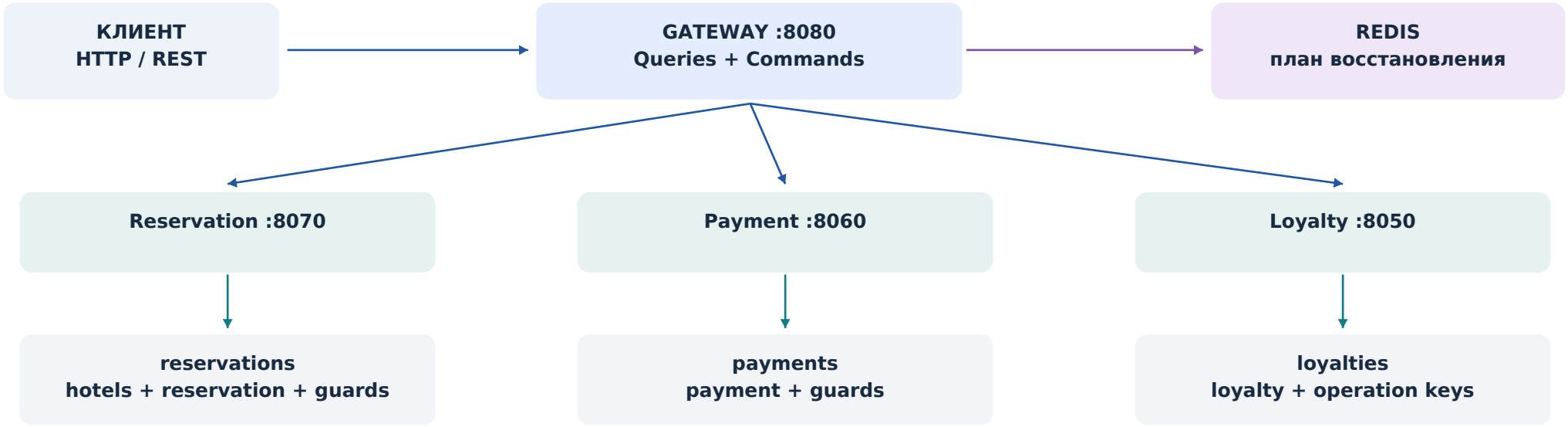
Как проходит запрос

Hotels Booking System · Gateway · CQRS · Saga · Redis



Между downstream-сервисами нет прямых вызовов. Gateway координирует сценарий; каждый сервис владеет своей логической БД.

КАК ЧИТАТЬ СХЕМЫ

Шаги нумеруются сверху вниз. Стрелка — вызов; «→ 200/201/204» в подписи объединяет вызов и ожидание ответа. Петля — локальная работа. Параллелизм указан явно.

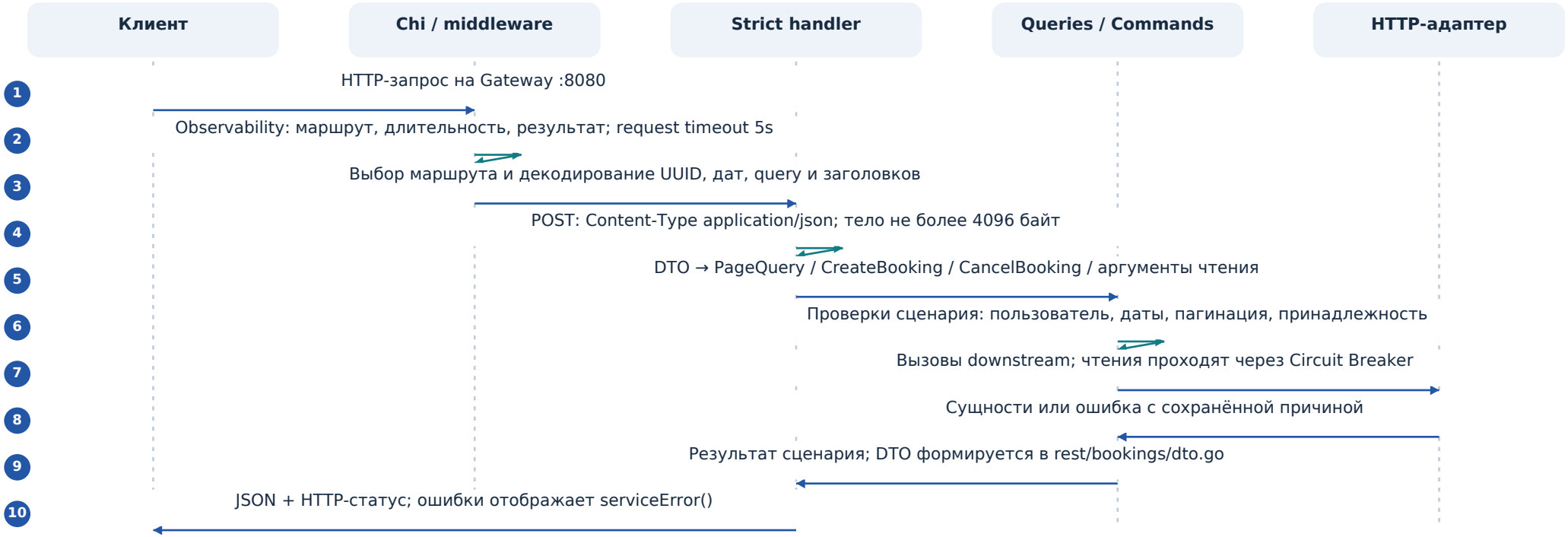
НАВИГАЦИЯ ПО PDF

С. 2 — HTTP-путь
С. 3–8 — чтения и создание
С. 9–11 — компенсация и отмена
С. 12–16 — операции, Redis, recovery
С. 17–19 — внутренние ручки сервисов
С. 20–24 — breaker, ошибки, служебные ручки, пример, код

Числа и порядок вызовов сверены с исходным кодом и YAML. Примеры пользователя Анна иллюстративны; health не проверяет downstream.

Сначала — общий HTTP-путь

Что происходит до и после любого бизнес-сценария



ПОРЯДОК И УСЛОВИЯ

1. Gateway разрешает в YAML только http.api_prefix=/api/v1. X-User-Name обязателен для пользовательских ручек; для списка отелей пользователь не нужен. Пустое имя, некорректный UTF-8 или длина > 80 символов отклоняются.

2. Ошибка декодирования даёт 400 ещё до usecase. Ограничение request timeout может закончить запрос до выполнения всей цепочки.

3. Queries и Commands собраны отдельно. Queries не имеет методов записи и доступа к очереди. Commands читает данные для расчёта и проверок.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 cmd/gateway.Run

02 rest.Observability → middleware.Timeout

03 api.HandlerWithOptions → api.NewStrictHandlerWithOptions

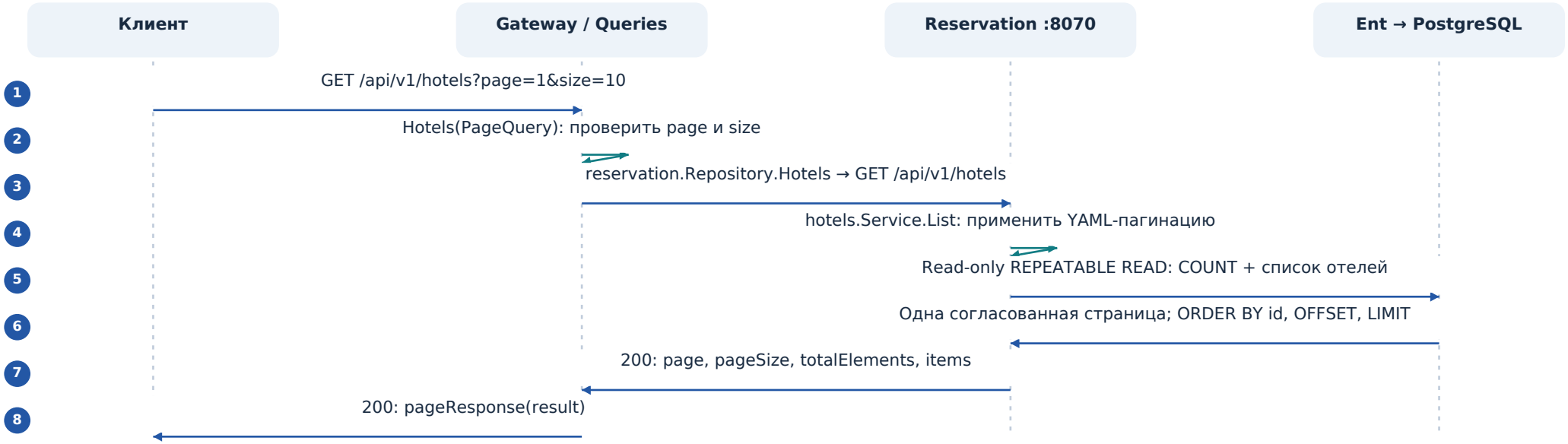
04 handler.Get… / Post… / Delete…

05 BookingQueries / BookingCommands

Код: services/gateway/cmd/gateway/run.go · services/gateway/internal/rest/bookings/register.go · services/gateway/internal/rest/bookings/handlers.go

01 · Список отелей

GET /api/v1/hotels?page=1&size=10 · успешный ответ 200



ПОРЯДОК И УСЛОВИЯ

1. Gateway принимает page ≥ 0; page=0 отправляет в Reservation как 1, а в ответе возвращает page=0. size в Gateway: 1..100.

2. Если параметры отсутствуют, Reservation использует default_page=1 и default_size=10 из своего YAML. Лимит Reservation отдельно равен 1000.

3. Отказ Reservation или открытый его breaker → 500 «Reservation Service unavailable». Payment, Loyalty и Redis не вызываются.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 handler.GetApiV1Hotels

02 Queries.Hotels

03 reservation.Repository.Hotels → httpclient.Read

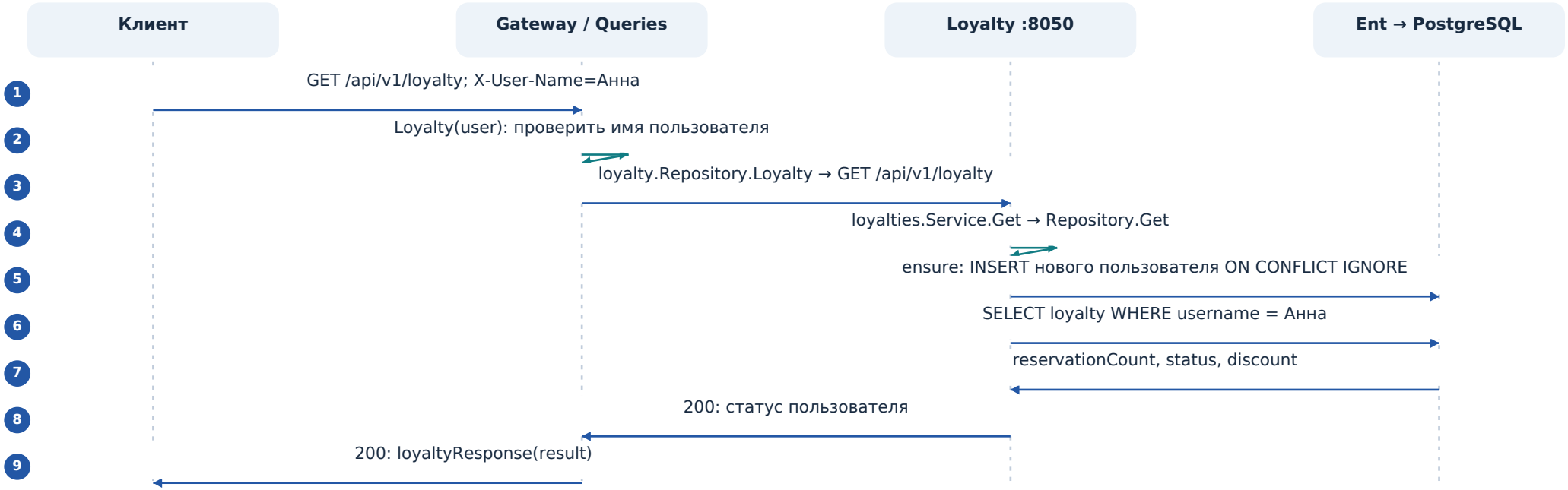
04 contracts.GetHotelsWithResponse

05 Reservation: hotels.Service.List → hotels.Repository.List

Код: services/gateway/internal/core/usecases/bookings/read.go · services/gateway/internal/reservation/repos/bookings.go · services/reservation/internal/core/usecases/hotels/service.go · services/reservation/internal/postgres/repos/hotels/repository.go

02 · Статус лояльности

GET /api/v1/loyalty · X-User-Name: Анна · успешный ответ 200



ПОРЯДОК И УСЛОВИЯ

1. Новый пользователь получает reservationCount=0, BRONZE, discount=5. Даже GET может создать строку Loyalty при первом обращении — это существующее поведение сервиса.

2. Для этой самостоятельной ручки fallback отсутствует: отказ Loyalty → 503, message «Loyalty Service unavailable».

3. Reservation, Payment и Redis не вызываются. Нулевой счётчик здорового ответа сохраняется как 0, а не исчезает.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 handler.GetApiV1Loyalty

02 Queries.Loyalty → validateUser

03 loyalty.Repository.Loyalty → httpclient.Read

04 contracts.GetLoyaltyWithResponse

05 Loyalty: Service.Get → Repository.Get → ensure + SELECT

Код: services/gateway/internal/core/usecases/bookings/read.go · services/gateway/internal/loyalty/repos/loyalty.go · services/loyalty/internal/postgres/repos/loyalties.go

03 · Все бронирования пользователя

GET /api/v1/reservations · запрос списка и параллельное обогащение



ПОРЯДОК И УСЛОВИЯ

1. Кэш отеля существует только внутри одного вызова List: sync.OnceValues разделяет результат и ошибку для одинакового H. Между HTTP-запросами кэша нет.

2. Hotel и Payment внутри одной строки вызываются последовательно; разные строки выполняются параллельно. Дедупликации Payment нет. N строк и U уникальных отелей: до 1+U+N чтений.

3. Ошибка Reservation/Hotel прерывает сборку списка; ошибка Payment доступности обнуляет только его блок. Ошибки 400/404 и отмена контекста не маскируются.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 handler.GetApiV1Reservations

02 Queries.List → reservations.Reservations

03 errgroup.SetLimit(enrichment_concurrency)

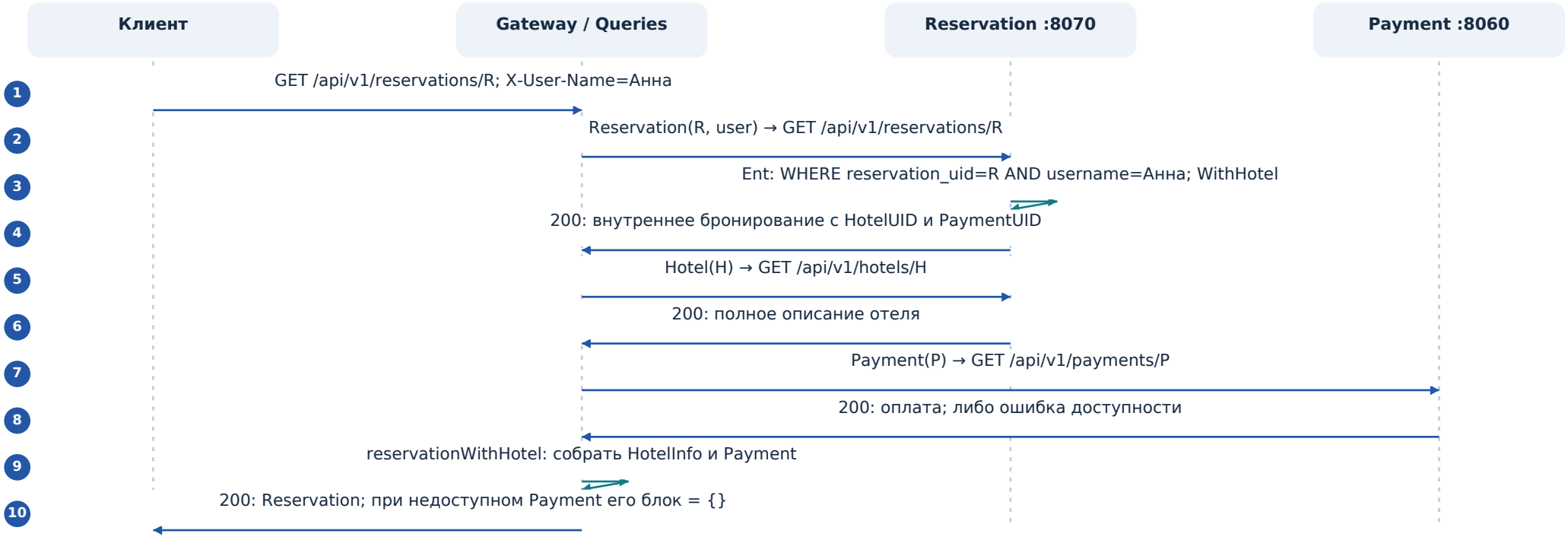
04 reservations.Hotel → reservationWithHotel → payments.Payment

05 reservationResponses(items)

Код: services/gateway/internal/core/usecases/bookings/list.go · services/gateway/internal/core/usecases/bookings/read.go · services/reservation/internal/postgres/repos/bookings/repository.go

04 · Одно бронирование

GET /api/v1/reservations/{reservationUid} · принадлежность проверяется в Reservation



ПОРЯДОК И УСЛОВИЯ

1. Три последовательных downstream-чтения. Внутренний ответ Reservation содержит ссылку на отель; Gateway дополнительно получает его полное описание.

2. Чужое или отсутствующее бронирование → 404. Проверка пользователя выполняется до чтения Payment. Неверный UUID пути → 400 при декодировании.

3. Loyalty и Redis не участвуют. Если Reservation недоступен, частичный успешный ответ невозможен → 500.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 handler.GetApiV1ReservationsReservationUid

02 Queries.Get → reservations.Reservation

03 reservations.Hotel

04 Queries.reservationWithHotel → payments.Payment

05 reservationResponse(result)

Код: services/gateway/internal/core/usecases/bookings/read.go · services/reservation/internal/postgres/repos/bookings/repository.go · services/gateway/internal/rest/bookings/dto.go

05 · Личный кабинет

GET /api/v1/me · сначала список, затем Loyalty



ПОРЯДОК И УСЛОВИЯ

1. Это последовательная композиция двух usecase-чтений: List → Loyalty. Между ними нет отдельной общей транзакции или общего снимка данных.

2. Payment может отсутствовать у отдельных броней. Loyalty может отсутствовать целиком. Если оба некритичных сервиса недоступны, ответ всё ещё возможен при здоровом Reservation.

3. Fallback действует только для ошибок доступности. Реальные нулевые значения здоровых моделей выводятся, 400/404 и отмена запроса не превращаются в пустой блок.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 handler.GetApiV1Me

02 Queries.Me

03 Queries.List → Hotel / Payment

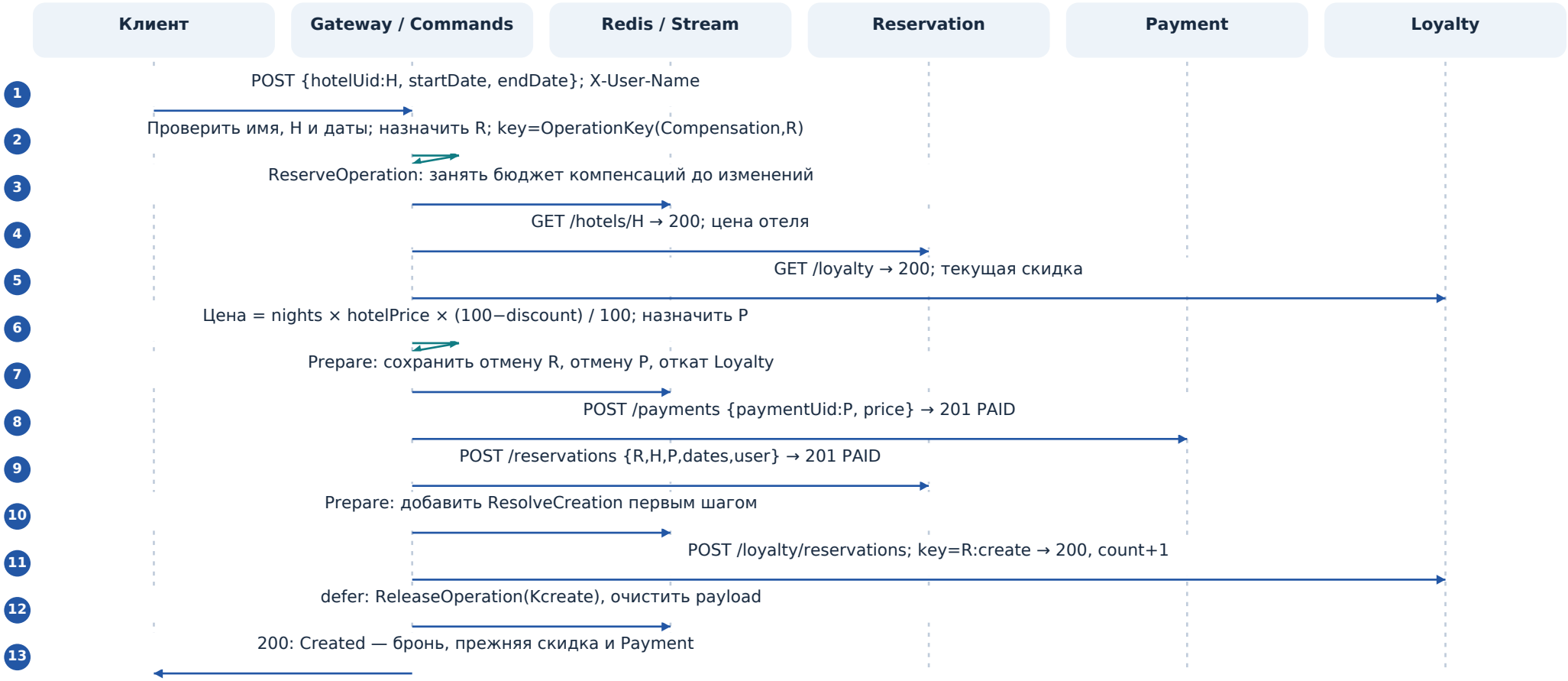
04 Queries.Loyalty

05 UserInfo → userInfoResponse

Код: services/gateway/internal/core/usecases/bookings/read.go · services/gateway/internal/core/usecases/bookings/list.go · services/gateway/internal/rest/bookings/dto.go

06 · Создание: успешный путь

POST /api/v1/reservations · синхронная Saga · внешний ответ 200



ПОРЯДОК И УСЛОВИЯ

1. Даты — календарные значения API. endDate должна быть позже startDate; стоимость вычисляется целочисленно, с защитой от переполнения.

2. Запись оплаты и бронирования PAID появляется до увеличения Loyalty. Скидка берётся до count+1. Количество свободных номеров не ограничивается.

3. Это несколько локальных транзакций разных сервисов. Ошибка Release тоже может повлиять на ответ; потерянный HTTP-ответ сам по себе не доказывает откат.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 handler.PostApiV1Reservations

02 Commands.Create(CreateBooking)

03 Groups.Create → Creation.CreatePayment / CreateReservation / IncreaseLoyalty

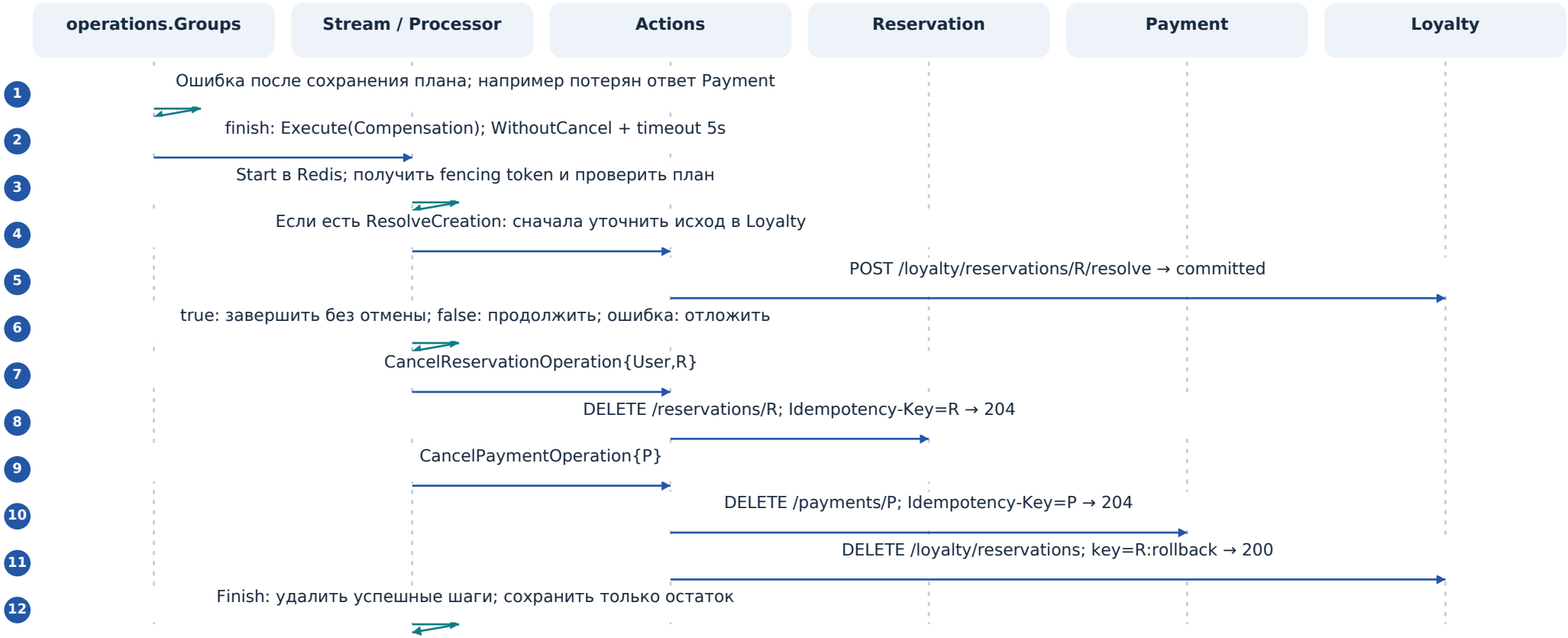
04 Payment.CreatePayment → Reservation.CreateReservation

05 Loyalty.ChangeLoyalty(+1) → ReleaseOperation

Код: services/gateway/internal/core/usecases/bookings/create.go · services/gateway/internal/core/usecases/operations/groups.go · services/gateway/internal/core/usecases/bookings/cost.go

Создание: если шаг не завершился

План сохранён до побочных эффектов; откат работает вне отменённого HTTP-контекста



ПОРЯДОК И УСЛОВИЯ

1. Для ошибки до увеличения Loyalty шага ResolveCreation ещё может не быть. CancelReservation, CancelPayment и RollbackLoyalty пробуются независимо: отказ первого не отменяет попытку второго.

2. Откат R:rollback уменьшает счётчик только если R:create действительно применился. Без увеличения записывается защита от позднего +1, чужие бронирования не уменьшаются.

3. Groups.Create преобразует panic в ErrOperationPanic: сохранённый план проходит через finish и компенсацию вне HTTP-контекста; ответ 500. Без плана освобождается резерв. POST Payment не повторяется автоматически.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 operations.Groups.Create → finish (defer)

02 Stream.Execute → worker.process(Initial)

03 Processor.compensate → operations.Actions.Execute

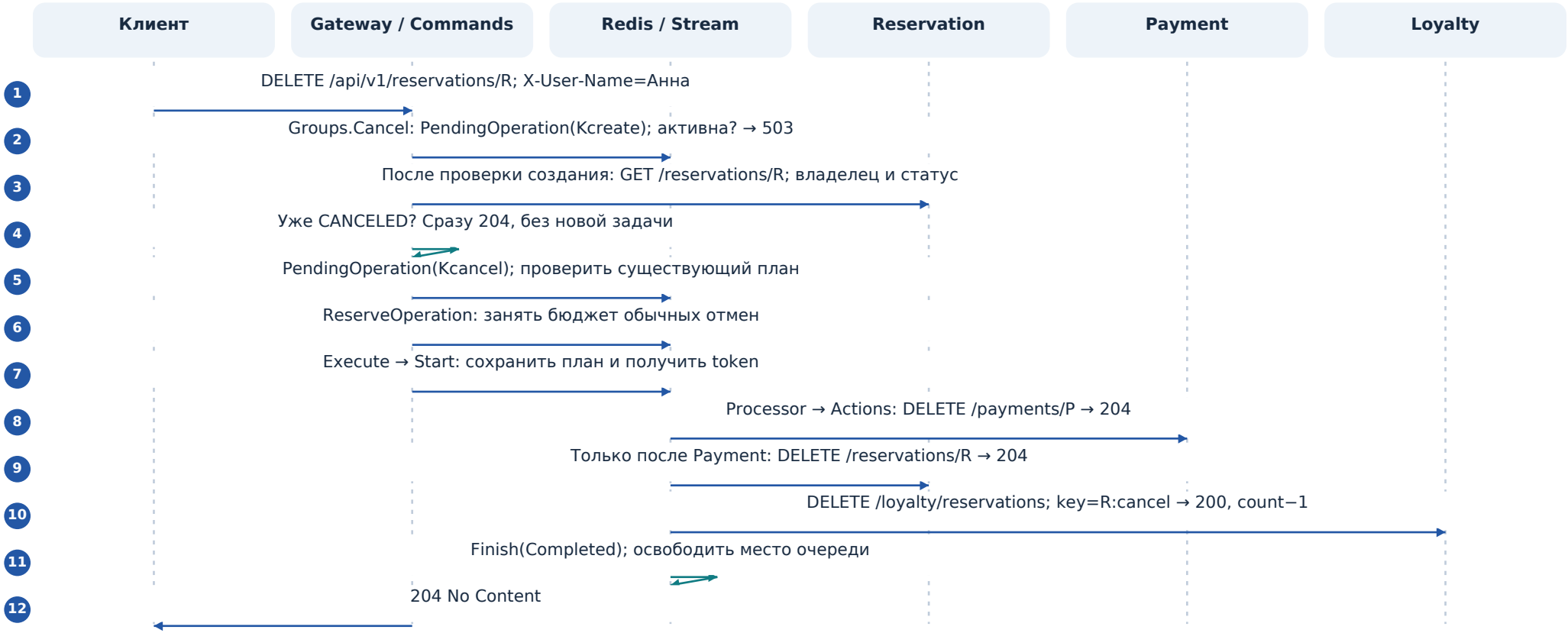
04 Actions.Execute → прямые вызовы HTTP-портов

05 Store.Finish: Completed / Deferred / Failed

Код: services/gateway/internal/core/usecases/operations/groups.go · services/gateway/internal/core/usecases/operations/compensate.go · services/gateway/internal/core/usecases/operations/actions.go

07 · Отмена: успешный путь

DELETE /api/v1/reservations/{reservationUid} · внешний ответ 204



ПОРЯДОК И УСЛОВИЯ

1. На диаграмме Stream обозначает цепочку worker.process → Processor.cancel → Actions; HTTP выполняют downstream-адаптеры Gateway, а не Redis.

2. Отмена последовательна: Payment → Reservation → Loyalty. Это другой порядок и другая политика ошибок, чем у компенсации создания.

3. Активное создание/восстановление → 503 без изменений. Затем читается актуальная бронь: отмена не обгоняет +1 Loyalty. Повтор R:cancel не уменьшает счётчик дважды.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 handler.DeleteApiV1ReservationsReservationUid

02 Commands.Cancel: validateUser → Groups.Cancel(user,R)

03 Groups.Cancel: pending(Kcreate) → Reservation → pending(Kcancel)

04 Groups.execute → reserve → Stream.Execute → Processor.cancel

05 Payment → Reservation → Loyalty → Store.Finish

Код: services/gateway/internal/core/usecases/bookings/cancel.go · services/gateway/internal/core/usecases/operations/cancel.go

Отмена: все важные развилки

Почему одинаковый DELETE может вернуть 204, 400, 404, 500, 502, 503 или 504

Ситуация	Данные / очередь	Внешний результат
Неверный ввод / чужая или отсутствующая бронь	Изменений нет	400 / 404
Reservation недоступен при чтении	Изменений нет	500
Redis недоступен / очередь заполнена	Бизнес-изменения ещё не начаты	503
Первичный отказ отмены Payment	Reservation PAID; слот освобождается	Ошибка, обычно 502 / 504
Payment отменён, Reservation не отменён	Остаток плана повторяется	Ошибка, обычно 500
P и R отменены, Loyalty недоступен	Остаётся DecreaseLoyalty; очередь принята	204
Все шаги выполнены / R уже CANCELED	Повторного уменьшения нет	204

ПОРЯДОК И УСЛОВИЯ

1. Повторный DELETE сначала читает Reservation. Если статус уже CANCELED, возвращает 204; ранее поставленная задача Loyalty продолжает выполняться независимо.

2. Если текущая задача ещё содержит CancelPayment или CancelReservation, повтор получает ошибку Reservation unavailable. Если остался только Loyalty, запрос принимается без повторного списания.

3. 204 после отказа Loyalty означает принятую отложенную работу, а не уже изменённый счётчик. Для критичных шагов успешный ответ не подделывается.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 Groups.Cancel → entities.HasAction[T](result.Task)

02 Processor.Execute: Initial vs Background

03 Initial + Payment не отменён → Rejected

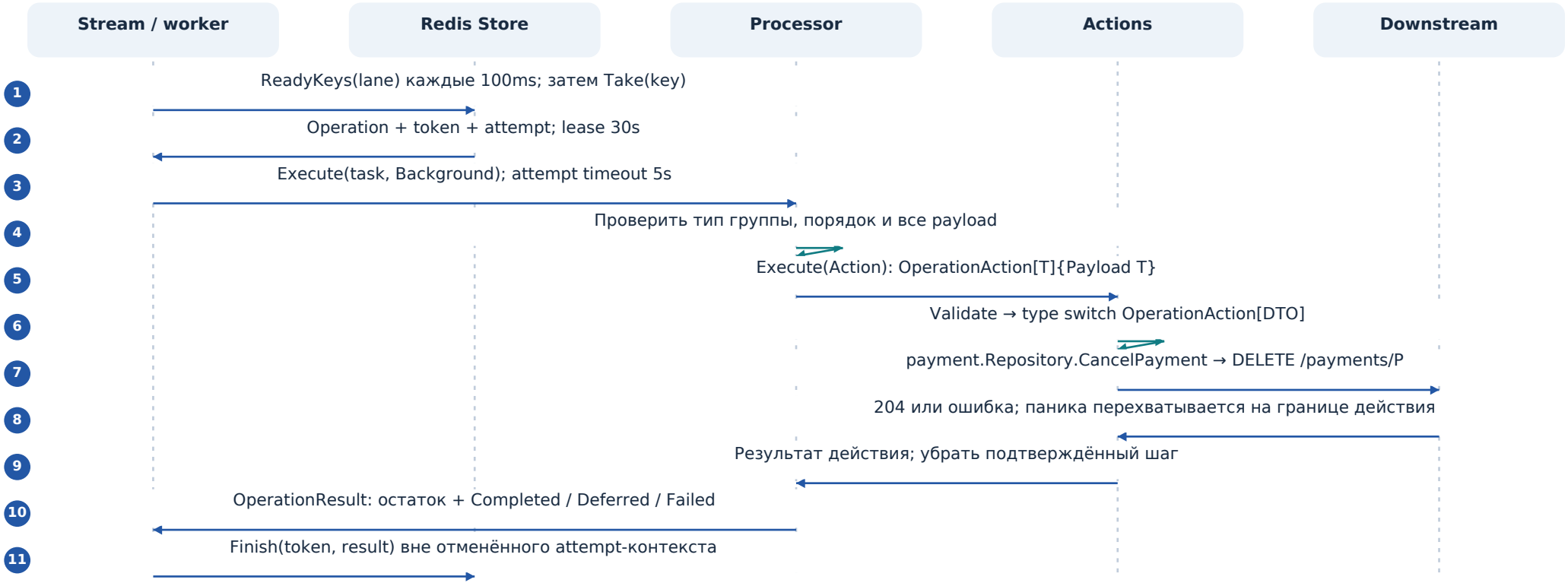
04 Deferred → план остаётся в Redis

05 serviceError → внешний статус

Код: services/gateway/internal/core/usecases/bookings/cancel.go · services/gateway/internal/core/usecases/operations/processor.go · services/gateway/internal/rest/bookings/errors.go

Как исполняется операция

У очереди свой UUID-ключ; бизнес-данные находятся в конкретных Go DTO



ПОРЯДОК И УСЛОВИЯ

1. Operation = {Kind, Key, Steps}. OperationAction[T] = {Payload T}; действие задаёт Go-тип DTO; Steps — []Action. Key — UUID задачи. DTO — Go-значение; только Redis-адаптер кодирует его в Gob. Конверт и действия не содержат версий.

2. Каждому действию соответствует свой DTO: CancelPayment, CancelReservation, ResolveCreation, RollbackLoyalty, DecreaseLoyalty. Processor зависит от интерфейса ActionExecutor; реализации связывает cmd/gateway.

3. Успешные действия не возвращаются в план. Повтор возможен, если действие выполнено, а Finish потерян; downstream защищает бизнес-эффект идемпотентностью.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 worker.runLane → runReady → execute

02 Store.Take → worker.process

03 Processor.Execute → cancel / compensate

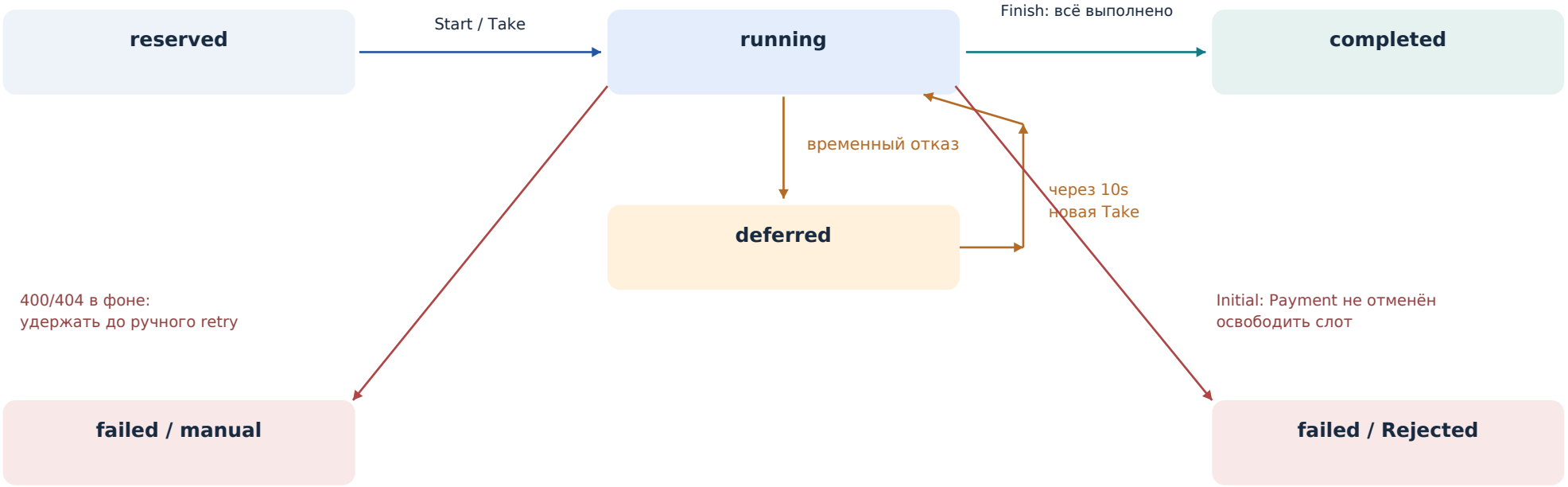
04 operations.Actions.Execute: type switch по Go-типу

05 operations.Actions → HTTP repo → Store.Finish

Код: services/gateway/internal/core/services/opstream/worker.go · services/gateway/internal/core/usecases/operations/actions.go · services/gateway/internal/models/entities/operation_payload.go

Redis: состояния и планирование

Redis Streams выдаёт задачи; Go координирует записи через WATCH + MULTI/EXEC



ПОРЯДОК И УСЛОВИЯ

1. Общий бюджет — 256 активных задач: 128 компенсаций и 128 отмен. Failed для ручного повтора удерживает место; Rejected освобождает слот. В обоих случаях Redis state=failed; различается удержание задачи.

2. Компенсации обслуживают 2 worker, отмены — 1, остаток только Loyalty — ещё 1. Общий параметр concurrency=4. Остаток отмены переводится в отдельную lane Loyalty.

3. Redis AOF с appendfsync always и отдельным volume сохраняет очередь. В PostgreSQL хранятся бизнес-данные и записи идемпотентности, а не эмуляция очереди.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 Reserve → reserved (запись может ещё не иметь payload)

02 Prepare → сохранить план до первого POST

03 Start / Take → running с lease и token

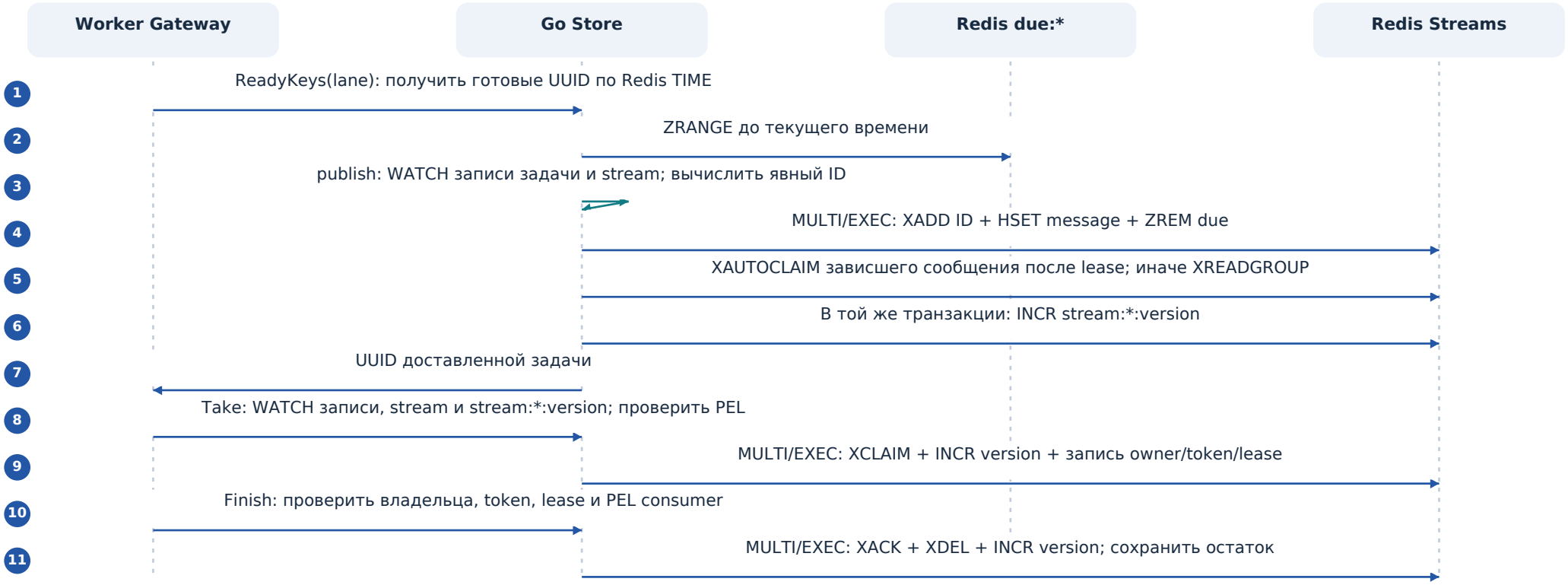
04 Deferred → следующий срок через 10s

05 Failed → inspect-operation / retry-operation

Код: services/gateway/internal/redis/repos/transaction.go · services/gateway/internal/redis/repos/streams.go · redis/redis.conf · services/gateway/config.yaml

Redis Streams: публикация и выдача

Sorted sets задают задержку; consumer group workers отслеживает доставку



ПОРЯДОК И УСЛОВИЯ

1. Изменения Pending Entries List (PEL) сами не инвалидируют WATCH. Поэтому XREADGROUP, XAUTOCLAIM, XCLAIM и подтверждение увеличивают stream:*:version в той же транзакции; Take и Finish наблюдают за этой версией.

2. Явный возрастающий ID позволяет атомарно записать сообщение и ссылку message в hash задачи. Finish при Deferred подтверждает старое сообщение и назначает новый срок в due:*.

3. Store.update повторяет только redis.TxFailedErr, в пределах storage_timeout. Сетевые ошибки не повторяются автоматически: исход EXEC может быть неизвестен. Lua и EVAL не используются.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 ReadyKeys → publish → XAUTOCLAIM / XREADGROUP

02 Take → deliveryOwned → XCLAIM + HSET

03 Finish → acknowledge → schedule / remove

Код: services/gateway/internal/redis/repos/streams.go · services/gateway/internal/redis/repos/transaction.go · services/gateway/internal/redis/repos/attempts.go

Падение Gateway: lease и fencing

Новый экземпляр подхватывает сохранённую задачу; старый не может перезаписать её результат



ПОРЯДОК И УСЛОВИЯ

1. Fencing защищает фиксацию очереди, но не отменяет уже отправленный HTTP-запрос старого worker. Повторный бизнес-эффект ограничивают ключи и транзакции downstream.

2. Если процесс умер после Reserve, но до Prepare, задача без payload после истечения lease освобождается: побочные эффекты ещё не должны были начаться.

3. Гарантии общей распределённой атомарности нет. Клиент также может не знать итог при потерянном ответе, даже если данные уже зафиксированы.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 Store.update: WATCH записи → проверки → MULTI/EXEC

02 Take: проверить PEL consumer, назначить token и attempt+1

03 Finish: owner + lease + token + state=running

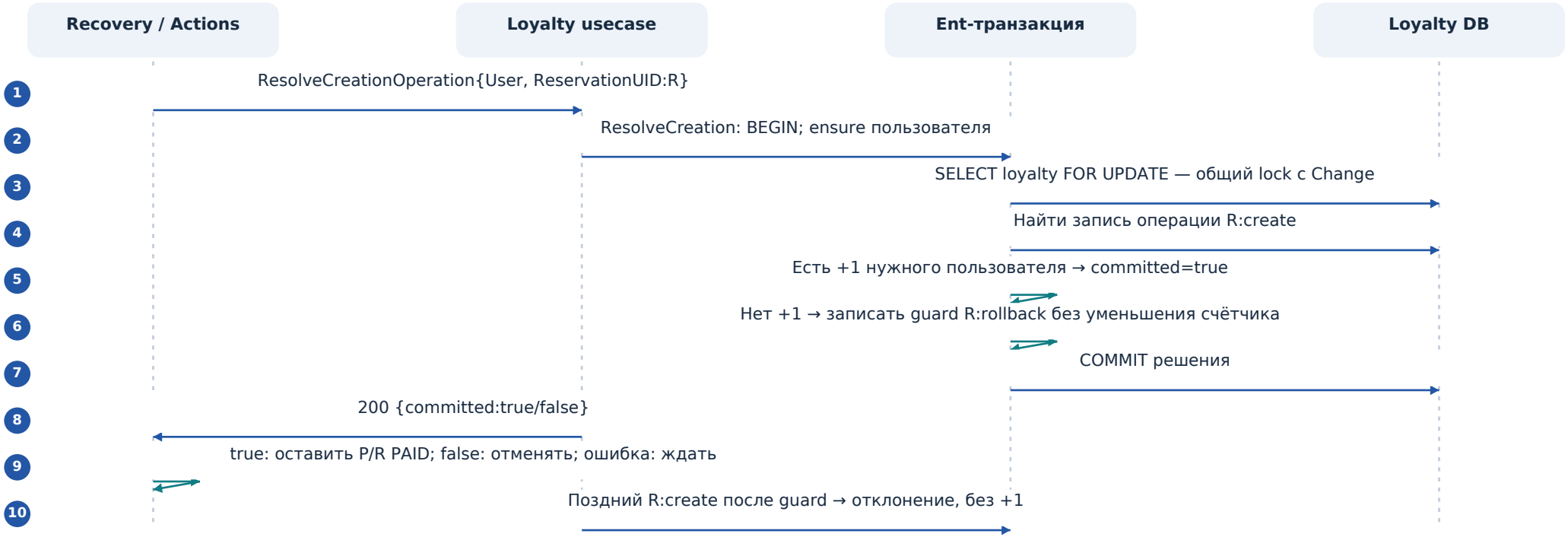
04 Payment/Reservation guards + Loyalty operation key

05 Gateway при shutdown останавливает worker и вызывает Drain

Код: services/gateway/internal/redis/repos/attempts.go · services/gateway/internal/core/services/opstream/worker.go · services/gateway/cmd/gateway/run.go

Неоднозначный исход создания

Что делать, если увеличение Loyalty закоммитилось, а ответ потерян



ПОРЯДОК И УСЛОВИЯ

1. ResolveCreation — изменяющий POST: он может записать запрет позднего создания. Он не защищается read-breaker и не является простым GET состояния.

2. Общий lock пользователя сериализует гонку ResolveCreation и Change(+1). Решение определяется зафиксированной записью R:create, а не предположением по таймауту HTTP.

3. Если Loyalty недоступен, компенсация с ResolveCreation не отменяет Payment и Reservation вслепую: весь план ждёт следующей попытки.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 Actions.Execute: ResolveCreationOperation

02 loyalty.Repository.ResolveCreation (HTTP)

03 Loyalty Service.ResolveCreation

04 postgres.Repository.ResolveCreation

05 CreationResolution.Decide под row lock

Код: services/gateway/internal/core/usecases/operations/compensate.go · services/loyalty/internal/postgres/repos/loyalties.go · services/loyalty/internal/models/entities/change.go

Внутренние ручки Reservation

Эти ручки вызывает Gateway; префикс /api/v1; сервис работает со своей логической БД

Метод / путь	Стек внутри Reservation	Работа с собственной БД
GET /hotels	rest/hotels → Service.List → Repository.List	REPEATABLE READ: COUNT + ORDER BY id / OFFSET / LIMIT
GET /hotels/{hotelUid}	rest/hotels → Service.Get → Repository.Get	Поиск hotels по hotel_uid; отсутствие → 404
GET /reservations	rest/bookings → Service.List → Repository.List	username=user; WithHotel; ORDER BY id
GET /reservations/{reservationUid}	rest/bookings → Service.Get → Repository.Get	reservation_uid + username; чужая бронь → 404
POST /reservations	rest/bookings → Service.Create → Repository.Create	Транзакция guard + сверка повтора + отель + INSERT PAID; 201
DELETE /reservations/{reservationUid}	rest/bookings → Service.CancelKnown / Cancel	guard canceled + UPDATE CANCELED; 204

ПОРЯДОК И УСЛОВИЯ

1. Create: claim BookingRequest(R,user), FOR UPDATE, проверить canceled/user, найти существующую бронь. Совпадающий повтор возвращает её; другой payload или статус → 400.

2. Новая бронь: найти отель в той же логической БД, создать reservation PAID со ссылкой на hotels.id, зафиксировать транзакцию. Reservation сам не вызывает Payment и Loyalty.

3. Gateway посылает DELETE с Idempotency-Key=R: CancelKnown сохраняет tombstone даже при ещё отсутствующей броне. Поздний POST с тем же R отклоняется. Без ключа сохранена legacy-семантика 404.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 Huma на Chi → REST DTO

02 coreifc.Bookings / Hotels → usecases

03 dbifc → Ent repositories

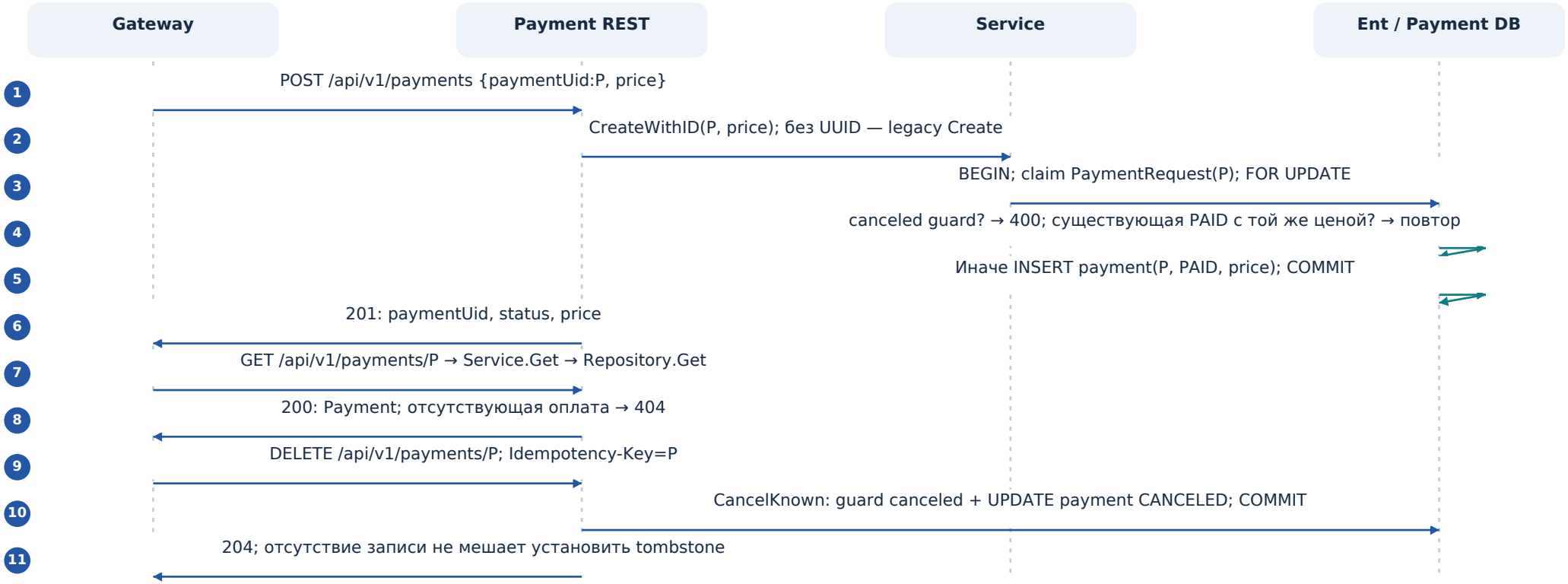
04 reservation + hotels + BookingRequest

05 COMMIT → сущность → REST DTO

Код: services/reservation/internal/rest/bookings · services/reservation/internal/rest/hotels · services/reservation/internal/postgres/repos/bookings/create.go · services/reservation/internal/postgres/repos/bookings/repository.go

Внутренние ручки Payment

GET — чтение; POST и DELETE используют request guard и локальную транзакцию



ПОРЯДОК И УСЛОВИЯ

1. Gateway заранее назначает P и проверяет совпадение UUID в ответе. Автоматического повтора POST Payment нет.

2. DELETE с неправильным ключом → 400. DELETE без ключа вызывает Cancel: сначала проверяет существование, поэтому неизвестный P → 404.

3. Это модель записи оплаты для лабораторной. Внешний платёжный провайдер в данном стеке не участвует. Payment не вызывает Reservation или Loyalty.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 rest/payments.Register (Huma)

02 payments.Service.CreateWithID / Get / CancelKnown

03 postgres.Repository.Create / Get / CancelKnown

04 PaymentRequest + payment, одна транзакция

05 Gateway contracts.ClientWithResponses проверяет статус

Код: services/payment/internal/rest/payments/register.go · services/payment/internal/postgres/repos/payments.go · services/gateway/internal/payment/repos/payments.go

Внутренние ручки Loyalty

Одна транзакция фиксирует ключ операции, счётчик, статус и скидку

Метод / путь	Сценарий	Что фиксируется
GET /loyalty	Service.Get → Repository.Get	ensure пользователя + SELECT; новый: 0 / BRONZE / 5
POST /loyalty/reservations	Service.Change(delta=+1)	R:create: count+1, статус и скидка; 200
DELETE /loyalty/reservations	Service.Change(delta=−1)	R:cancel или R:rollback; разные правила отката; 200
POST /loyalty/reservations/{R}/resolve	Service.ResolveCreation	Зафиксировать исход создания, вернуть committed; 200

ПОРЯДОК И УСЛОВИЯ

1. Change: BEGIN → ensure → SELECT пользователя FOR UPDATE → прочитать связанный ключ → claim текущего ключа → LoyaltyChange.Decide → UPDATE → COMMIT.

2. Повтор с тем же ключом и теми же user/delta возвращает текущее состояние без повторного изменения. Конфликт содержания ключа → 400. Запрос без ключа выполняется как legacy-изменение.

3. R:rollback проверяет R:create: если увеличения не было, реальный delta=0; guard записывается. R:cancel уменьшает count на 1, но не ниже 0.

4. Статус рассчитывается по новому счётчику: <10 BRONZE/5%; 10..19 SILVER/7%; ≥20 GOLD/10%. Правила находятся в entities, блокировки и фиксация — в Ent-репозитории.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 rest/loyalties.Register

02 Service.Change → NewLoyaltyChange

03 Repository.Change: транзакция + FOR UPDATE

04 LoyaltyChange.Decide(current, applied, related)

05 COMMIT key + count + status + discount

Код: services/loyalty/internal/rest/loyalties/register.go · services/loyalty/internal/models/entities/change.go · services/loyalty/internal/postgres/repos/loyalties.go

Circuit Breaker и fallback

У каждого downstream свой breaker; он охватывает законченные чтения



успех пробы → CLOSED; availability-ошибка пробы → снова OPEN

ПОРЯДОК И УСЛОВИЯ

1. Closed: пять последовательных ошибок доступности открывают breaker. Open: в течение 1s чтения сразу получают unavailable, без сетевого вызова. Half-open: допускается одна проба.

2. Успешная проба закрывает breaker; новая ошибка доступности снова открывает. Исключены отмена клиентом и истечение его контекста; 400/404 не считаются ошибкой доступности.

3. Успех определяется после чтения и разбора ответа, а не только получения HTTP-заголовков. Записи POST/DELETE, включая ResolveCreation, не проходят через breaker.

4. Fallback — пустой JSON-блок, а не чтение старого значения из Redis. При неотменённом контексте он применяется только к availability-ошибкам.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 Repository.Payment / Loyalty / Hotel / Reservation / Hotels

02 httpclient.Read → breaker.Allow

03 Вызов HTTP + проверка статуса + декодирование

04 done(success / availability failure / excluded)

05 Queries решает: fallback или ошибка

Код: services/gateway/internal/httpclient/breaker.go · services/gateway/internal/models/entities/availability.go · services/gateway/internal/core/usecases/bookings/read.go

Статусы и ограничения

Как ошибка превращается в ответ Gateway; параметры взяты из YAML

Условие	Ответ / правило	Где применяется
Неверный ввод / отсутствующий объект	400 / 404	Валидация REST и usecase; ownership в Reservation
Reservation недоступен	500: Reservation Service unavailable	Все критичные чтения и сценарии изменений
Loyalty недоступен	503: Loyalty Service unavailable	Самостоятельное GET и создание бронирования
Payment недоступен при чтении	200 с payment={}	GET reservations, GET reservations/R, GET me
Loyalty недоступен в /me	200 с loyalty={}	Только если список удалось собрать
Redis недоступен / занят / полон	503	До побочных эффектов резервируется слот
Прочий downstream отказ / timeout	Обычно 502 / 504	Если более приоритетная service-ошибка не определена
HTTP Gateway / downstream	5s / 2s	2s на одно обращение, общий запрос ограничен 5s
Параллелизм / in-flight	4 строки; Reservation 16, Payment 8, Loyalty 8	Ограничены worker обогащения и HTTP-клиенты

ПОРЯДОК И УСЛОВИЯ

1. Mapper проверяет errors.Is в определённом порядке. Поэтому критичный Reservation timeout обычно становится 500, а Loyalty timeout — 503, а не универсальным 504.

2. Пять секунд — общий request timeout, а не обещание, что последовательность из нескольких двухсекундных вызовов всегда уложится. Detached-компенсация отдельно ограничена 5s.

3. При недоступности Payment и Loyalty здоровые нулевые поля не исчезают. Статус неизвестного блока определяет преобразование DTO, а не omitempty по цене или счётчику.

Код: services/gateway/internal/rest/bookings/errors.go · services/gateway/internal/rest/bookings/dto.go · services/gateway/config.yaml

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 serviceError: panic → invalid → not found

02 → Loyalty → Reservation → очередь

03 → timeout → upstream unavailable

04 dto.paymentResponse / loyaltyResponse

05 YAML определяет лимиты и таймауты

Служебные ручки

Health, метрики и документация не запускают бронирование или Saga

Ручка Gateway	Что происходит после запроса	Что получает клиент
GET /manage/health	Chi handler выставляет статус; downstream не опрашиваются	200, без тела
GET /metrics	telemetry.Handler собирает Prometheus collectors	Метрики HTTP, Go/process и операций; без user/UUID labels
GET /api/v1/docs	Huma публикует Swagger UI для сгенерированного API	HTML интерфейс документации
GET /api/v1/openapi.json	JSON схемы из api.GetSpec, сериализованной при запуске	OpenAPI-контракт Gateway

ПОРЯДОК И УСЛОВИЯ

1. Текущие пути взяты из YAML. Документация и метрики могут быть переименованы конфигурацией. Здоровье Gateway не означает доступность Reservation, Payment или Loyalty.

2. Каждый downstream также имеет GET /manage/health. Служебные endpoints downstream регистрируются общим infrastructure-кодом; конкретные docs/metrics пути задаются YAML сервиса.

3. Docker entrypoints принимают --config PATH и --config=PATH в любом месте до --; Atlas и приложение используют один YAML. Store работает с UUID-ключами и Gob-конвертом без версий. Преобразования старых данных нет. При запуске собираются клиенты, три breaker, Actions, Processor, Redis Store, Stream, Commands и Queries. Worker стартует рядом с HTTP-сервером.

4. При завершении HTTP-сервер прекращает приём и завершает запросы; затем останавливаются worker, вызывается Drain с лимитом 20s, закрываются Redis и остальные ресурсы.

Код: services/gateway/internal/rest/system/register.go · services/gateway/cmd/gateway/run.go · modules/platform/rest/server.go

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 cmd/gateway.Run

02 system.Register → api.GetSpec / Huma / telemetry

03 router.Get("/manage/health", …)

04 rest.Serve → server.Shutdown

05 stopStream → stream.Run finished → Drain

Конкретный пример: Анна

Три ночи × 10 000 ₽; начальный счётчик 9; читаем, создаём и отменяем

До создания
9 / BRONZE / 5%

→

Создание R
28 500 ₽; PAID

→

После +1
10 / SILVER / 7%

→

После отмены
9 / BRONZE / 5%

ПОРЯДОК И УСЛОВИЯ

1. 1. GET /loyalty: count=9, BRONZE, discount=5. GET /hotels возвращает отель Ararat Park Hyatt Moscow с ценой 10 000.

2. 2. POST с 2026-10-20 по 2026-10-23: nights=3; цена 30 000×95/100=28 500. Созданы Payment P и Reservation R со статусом PAID.

3. 3. R:create увеличивает счётчик до 10: теперь SILVER/7%. Ответ этого создания всё ещё содержит discount=5 — именно эта скидка применена к оплате.

4. 4. GET /me собирает R + Hotel + Payment(28 500), затем текущее Loyalty(10, SILVER, 7). Скидка в лояльности уже может отличаться от скидки прошлого бронирования.

5. 5. DELETE R: P→CANCELED, затем R→CANCELED, затем R:cancel: count 10→9, BRONZE/5. Повтор DELETE даёт 204 без второго уменьшения.

6. 6. Если Loyalty выключен только при отмене, DELETE даёт 204 после принятия остатка задачи. Счётчик изменится после восстановления сервиса и повтора worker.

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА

01 POST /api/v1/reservations

02 X-User-Name: Анна

03 Content-Type: application/json

04 {"hotelUid":"049161bb-badd-4fa8-9d90-87c9a82b0668",

05 "startDate":"2026-10-20", "endDate":"2026-10-23"}

06 Проверка результата: GET /api/v1/me

Код: services/gateway/internal/core/usecases/bookings/create.go · services/loyalty/internal/models/entities/loyalty.go · services/gateway/internal/core/usecases/bookings/cancel.go

Куда смотреть в коде

Навигатор по реализации и границы описания

Вопрос	Файл / каталог
Как собраны зависимости и жизненный цикл	services/gateway/cmd/gateway/run.go
HTTP → DTO → конкретный usecase	services/gateway/internal/rest/bookings/handlers.go
Чтение списка, одной брони, /me, отелей и Loyalty	services/gateway/internal/core/usecases/bookings/{read,list}.go
Создание и отмена	services/gateway/internal/core/usecases/bookings/{create,cancel}.go
Фоновый запуск и фиксация	services/gateway/internal/core/services/opstream/{stream,worker}.go
План, порядок и политика повторов	services/gateway/internal/core/usecases/operations
Типы payload / маршрутизация	services/gateway/internal/models/entities/operation_payload.go; internal/core/usecases/operations/actions.go
Выдача Streams, транзакции и fencing	services/gateway/internal/redis/repos/{streams,transaction,attempts}.go
Транзакции, guards и счётчик	services/{reservation,payment,loyalty}/internal/postgres/repos
Лимиты и контракты	services/gateway/config.yaml; internal/clients/contracts/openapi.yaml

ПОРЯДОК И УСЛОВИЯ

1. Схемы отражают исходный код на дату сборки документа. Они описывают возможные вызовы и ветвления, а не фактическую трассу конкретного запущенного запроса.

2. Гарантии: отдельные локальные транзакции, сохранённый план, идемпотентные действия и fencing фиксации очереди. Общей транзакции Redis + три БД и гарантии exactly-once доставки HTTP нет.

3. Изменения файлов приложения этим документом не требуются. Пересборка PDF и Markdown: python3 scripts/render-gateway-guide.py (нужен ReportLab и шрифт DejaVu Sans).

Код: docs/services.md · docs/sequences.md · scripts/render-gateway-guide.py

СТЕК ВЫЗОВОВ / ТОЧКИ КОДА